Comparing Heuristics and Machine Learning for Wordle Solving

Justin Hoffman
School of Information Technology
Illinois State University
IT 448 Graduate Machine Learning, Spring 2026

ABSTRACT

This paper compares heuristic and machine learning approaches for solving Wordle, a combinatorial word-guessing game whose generalized decision problem is NP-hard [1]. Six solvers spanning heuristic search, minimax partitioning, reinforcement learning, and approximate dynamic programming were implemented and evaluated exhaustively over the 2,315-word answer set under a closed-vocabulary constraint.

A cached state rollout solver achieved 3.477 average guesses, approaching the 3.421 optimum computed by Bertsimas and Paskov [2]. It also shows that effective play traverses only 331 distinct game states, suggesting that practical complexity is far lower than worst-case hardness implies.

A teacher-guided Deep Q-Network reached a 97.7% win rate at 3.678 average guesses, substantially outperforming the baseline DQN (67.2%) and approaching Ho's reported results [3]. Performance was limited by misalignment between the reward function and the true objective of candidate-set reduction, as well as reliance on external solver logic at inference. Planning methods outperformed learned policies overall, but guided reinforcement learning approached planning-level performance when problem structure was effectively exploited.

CCS CONCEPTS

• *Computing methodologies → Reinforcement learning* • *Computing methodologies → Heuristic function construction* • *Computing methodologies → Planning for deterministic actions*

KEYWORDS

Wordle, reinforcement learning, deep Q-network, rollout, dynamic programming, heuristic search, minimax, imitation learning

1 INTRODUCTION

Wordle is a word-guessing game in which a player has six attempts to identify a hidden five-letter word. After each guess, tile-colored feedback indicates which letters are correct and correctly placed (green), correct but misplaced (yellow), or absent (gray). Despite these simple rules, Lokshtanov and Subercaseaux [1] show that a natural formalization of Wordle is NP-hard: given a dictionary and a guess limit, it is intractable to determine whether a winning strategy exists. This hardness persists for five-letter words and extends to approximation, providing formal justification for heuristic, planning, and learning-based methods.

Bertsimas and Paskov [2] computed an exact optimal policy via dynamic programming, achieving a 100% win rate with an average of 3.421 guesses and a maximum of five. Their analysis identifies SALET as the optimal first guess under an open-vocabulary formulation in which guesses may be drawn from the full set of valid Wordle inputs. Many practical solvers, including those in this study, restrict guesses to the 2,315-word answer list, so direct comparisons must be interpreted carefully.

This project originated from a practical challenge. In 2022, prior to formal study of information theory or machine learning, the author developed a Python-based Wordle solver ("Wordmaster Master") using letter-frequency heuristics. Its strong empirical performance raised a central question: can reinforcement learning outperform carefully designed heuristics in this structured combinatorial setting?

The project was further motivated by Ho [3], who introduced a 417-dimensional state representation and a 130-dimensional factorized output scored via dot product against word encodings. Using staged training across progressively larger vocabularies and millions of simulated games, Ho reported a 98% win rate at 4.1 average guesses, with A2C outperforming DQN at scale. This paper implements six Wordle solvers evaluated exhaustively over the 2,315-word answer set, enabling controlled comparison across heuristic, planning-based, and reinforcement learning approaches.

2 RELATED WORK

Lokshtanov and Subercaseaux [1] established that a generalized formulation of Wordle is NP-hard, even for five-letter words, providing theoretical justification for approximate solution methods. Liu [4] compared frequency scoring, entropy maximization, and KL-divergence heuristics as classroom examples, reporting 3.674 average guesses with his best method. Chen et al. [10] proposed a two-phase entropy-based optimization approach that explicitly maximizes information gain when selecting guesses, further demonstrating the effectiveness of information-theoretic heuristics in Wordle-like games. Bhambri, Bhattacharjee, and Bertsekas [5] modeled Wordle as a partially observable Markov decision process and

applied rollout-based approximate dynamic programming, achieving performance within 0.4% of optimal.

Learning-based approaches have taken several forms. Anderson and Meyer [6] applied tabular Q-learning to pick among heuristic strategies rather than individual words, showing that meta-policies over abstracted actions can still capture useful decision structure. Weng and Feng [7] took a statistical approach, using neural networks to model guess distributions based on features such as letter frequency and character uniqueness.

Two methodological works inform the deep RL components of this study. Ross, Gordon, and Bagnell [8] introduced the DAgger algorithm for imitation learning, showing that training on the learner's own state distribution mitigates compounding errors. Mnih et al. [9] proposed the Deep Q-Network framework with experience replay and target networks, which form the foundation for the deep reinforcement learning methods used here.

3 METHODOLOGY

3.1 Game Engine and Evaluation

All solvers interact with a shared Wordle environment that implements official feedback generation and candidate filtering. Repeated letters are handled with a two-pass procedure: green matches are assigned first, then yellow matches are drawn from the remaining unmatched target letters.

Each solver is evaluated over all 2,315 answer words. Performance is summarized by win rate and average guesses. Total runtime over the full evaluation set is measured as a rough proxy for per-game inference cost.

3.2 Frequency Heuristic (Solver 1)

The frequency solver, originally developed in 2022, scores candidate words by combining global letter frequency and positional frequency over the current candidate set S_t. Global frequency counts how many remaining words contain a given letter; positional frequency counts how many have that letter at a specific position.

The score for a candidate word $w = (w_1, \dots, w_5)$is:

$$\text{score}(w) = \sum_{i=1}^{5} \left(f_{\text{global}}(w_i) + 3 \cdot f_{\text{pos}}(w_i, i) \right) \tag{1}$$

where

$$f_{\text{global}}(l) = \mid \{w' \in S_t : l \in \{w'_1, \dots, w'_5\}\} \mid$$

and

$$f_{\text{pos}}(l, i) = \mid \{w' \in S_t : w'_i = l\} \mid.$$

Letters already confirmed green are excluded from scoring to avoid double-counting known information. When the candidate set is large (e.g., $\mid S_t \mid > 2$), the solver also considers exploratory guesses drawn from the full vocabulary, picking words that maximize expected information gain even if they cannot themselves be the answer.

3.3 Minimax Partition Solver (Solver 2)

For each candidate guess $g \in S_t$, feedback is simulated against all remaining candidate words $w \in S_t$. The resulting candidate set is partitioned by feedback pattern, and the guess minimizing the size of the largest partition is selected:

$$g^* = \arg\min_{g \in S_t} \max_{f} \mid \{w \in S_t : \text{feedback}(g, w) = f\} \mid \tag{2}$$

where f ranges over all possible feedback patterns. This minimizes the worst-case candidate set size at each step and is closely related to information-theoretic strategies for reducing uncertainty. The optimal opening guess is computed once and cached for reuse across games.

3.4 Tabular Q-Learning (Solver 3)

Following Anderson and Meyer [6], this approach applies tabular Q-learning at the level of strategy selection rather than word selection. The agent learns a meta-policy over five predefined strategies:

1. **Random**: uniform random selection from the allowed vocabulary.
2. **Curated list**: selection from a fixed set of roughly 1,600 high-quality guesses, built by running the frequency solver over the full evaluation set and collecting every word it used in the first three turns.
3. **Frequency-based**: the word maximizing the scoring function in Equation (1).
4. **Constraint-based**: a guess from the current candidate set satisfying all known positional and letter constraints.
5. **Exclude-only**: a guess prioritizing untested letters without enforcing positional constraints or restricting to the current candidate set, which focuses on shrinking the search space rather than identifying the target directly.

Together, these options span a range of tradeoffs between exploration and exploitation.

Each state is defined by the number of green and yellow tiles from the previous guess, yielding 19 reachable states. Training uses standard Q-learning under an ε-greedy policy with $\varepsilon = 0.3$. Over 10,000 episodes, the model converges to a lookup table mapping states to preferred strategies.

At inference, the learned policy selects a strategy given the current state, and that strategy produces the next guess. This decomposition shrinks the action space dramatically but caps optimality at the quality of the predefined strategies.

3.5 Rollout Solver (Solver 4)

Following Bhambri, Bhattacharjee, and Bertsekas [5], the rollout solver is treated as a reinforcement learning and adaptive control method applied to a partially observable decision problem. Wordle is modeled as a partially observable Markov decision process (POMDP) in which the hidden word is unknown. The agent acts on observed feedback while implicitly maintaining a belief over the candidate set S_t, with the objective of minimizing the expected number of guesses to reach a terminal state.

Rollout methods approximate optimal policies by pairing a base policy with one-step lookahead. Rather than solving the full dynamic program, each candidate action is evaluated by simulating its outcome and then following the base policy. The action with the lowest expected cost is selected.

The base policy here is the frequency solver from Section 3.2. At each step, a shortlist of candidate guesses is ranked by frequency score when $|S_t|$is large, switching to the minimax criterion in Equation (2) as the set shrinks. For each candidate g, the solver simulates the game forward against all remaining hidden words $w \in S_t$, applies the base policy after the initial action, and averages the resulting guess counts. The candidate minimizing expected cost is selected.

Results are cached by state, where each state is defined by the current candidate set S_t. Despite the large theoretical state space implied by the NP-hard formulation, only 331 unique states were encountered during exhaustive evaluation. Subsequent evaluations then reduce to fast lookups.

3.6 Deep Q-Network v1 (Solver 5)

The Deep Q-Network (DQN) learns a policy by approximating the action-value function $Q(s, a)$, which estimates the expected long-term reward of selecting guess $a$in state $s$and continuing optimally [9]. Unlike the heuristic and planning-based solvers, the model must infer effective strategies from experience, making performance highly dependent on reward design and exploration.

The game state is encoded as a 417-dimensional vector following Ho [3], with three components: a letter-position-status tensor capturing feedback history, global absent-letter flags, and a normalized turn indicator. Formally,

$$\dim(s) = L \cdot P \cdot S + L + 1 \qquad (3)$$

where $L = 26$, $P = 5$, and $S = 3$are the number of letters, positions, and feedback states (gray, yellow, green). The first term ($L \cdot P \cdot S = 390$) encodes which feedback color each letter has received at each position. The next $L = 26$features flag letters inferred to be absent, and the final feature encodes the normalized turn number.

The encoded state is passed through a multilayer perceptron

$$417 \rightarrow 512 \rightarrow 512 \rightarrow 130 \qquad (4)$$

with ReLU activations. Rather than outputting a value for each word in the vocabulary, the network produces a 130-dimensional vector corresponding to letter-position pairs. Each candidate word $w$is encoded as a 130-dimensional one-hot vector $e(w)$, and its value is computed via

$$Q(s, w) = z(s)^{\top} e(w) \qquad (5)$$

where $z(s)$is the network output. This factorization enables generalization across words but assumes word quality decomposes into independent letter-position contributions.

At inference, all candidate words are scored using Equation (5) and the highest-valued guess is selected. Previously guessed words are masked, and if only one candidate remains it is returned directly. These constraints improve stability but mean the solver is not fully end-to-end learned.

Training uses Double DQN to reduce overestimation bias, with target

$$y = r + \gamma \max_{a'} Q(s', a') \qquad (6)$$

Experience replay (100,000 transitions), target network updates every 500 episodes, gradient clipping (max norm 1.0), and epsilon-greedy exploration decayed from 1.0 to 0.05 stabilize learning.

The reward function assigns +2 for green tiles, +1 for yellow, +10 plus a speed bonus for solving, and -10 for failure. This provides dense feedback but is misaligned with the true objective of reducing the candidate set, so strong-looking guesses may be strategically poor while informative exploratory guesses may be undervalued.

3.7 Teacher-Guided DQN v2 (Solver 6)

DQN v2 uses the same architecture as v1 but replaces random exploration with teacher-guided exploration from the rollout solver. During exploration, the agent follows the teacher's action. During exploitation, a +20 reward bonus is applied when the selected action matches the teacher, pushing the policy toward rollout-based decisions. Training runs for 20,000 episodes.

This is related to DAgger [8], since expert guidance is provided under the learner's evolving state distribution, though it is not a direct implementation. At inference, the same masking and single-candidate shortcuts used in v1 apply, producing a hybrid learned and rule-based policy.

4 RESULTS

Table 1: Performance Comparison (2,315-word benchmark)

Solver	Type	Win %	Avg. Guesses	Speed its/s
Optimal [2]	Exact DP	100.0	3.421	—
Rollout	DP/Plan	100.0	3.477	182.7
Frequency	Heuristic	100.0	3.575	19.5
Minimax	Heuristic	100.0	3.644	2.6
Tabular Q	RL	99.0	3.651	70.5
DQN v2 (Teacher)	Deep RL	97.7	3.678	135.6
DQN v1 (Pure)	Deep RL	67.2	4.582	144
Ho [3] (reported)	Deep RL	~98	~4.1	—

Table 1 presents complete evaluation results over the 2,315-word benchmark. The rollout solver achieved the best performance at 3.477 average guesses with a 100% win rate, within 1.6% of the 3.421 optimum reported by Bertsimas and Paskov [2]. The frequency heuristic, developed before any formal machine learning work, also achieved 100% at 3.575 average guesses, outperforming every learning-based approach.

The minimax solver averaged 3.644 guesses, slightly worse than the frequency heuristic. Minimizing worst-case partition size does not necessarily minimize expected guess count.

Among reinforcement learning methods, tabular Q-learning reached 99.0% at 3.651 average guesses by learning a policy over strategy selection rather than individual words. The teacher-guided DQN (v2) reached 97.7% at 3.678, substantially improving over the baseline DQN and approaching Ho's reported results, though direct comparison is limited by differences in vocabulary and training setup.

The contrast between DQN v1 (67.2%) and DQN v2 (97.7%) is the most significant result. Both models share identical architectures; the only difference is the quality of exploration and training signal. DQN v1 relies on random exploration, which produces low-quality trajectories. As exploration decreases, the model increasingly trains on its own suboptimal behavior and performance degrades. DQN v2 uses rollout-based teacher guidance, so the replay buffer contains high-quality state-action pairs. This is consistent with Ho [3], who found that standard DQN struggled at scale while alternative approaches with better training dynamics performed more reliably.

5 DISCUSSION

5.1 Planning vs. Learning

A central finding is that despite the NP-hardness of the generalized Wordle problem [1], the effective belief-state space under near-optimal play is extremely small. The rollout solver encountered only 331 distinct states during exhaustive evaluation over the 2,315-word answer set, where each state corresponds to a reachable candidate set. From the initial state, each guess partitions the candidate set into equivalence classes under feedback, and policies that optimize expected guess count visit only a narrow subset of these partitions.

This gap between worst-case and typical-case complexity explains why planning performs well in practice. Rollout constructs an approximate dynamic program over visited states, caching the one-step lookahead value for each encountered state and reusing it when the state recurs, which amortizes computation across the benchmark.

Even so, explicit planning outperformed learned policies. Wordle's deterministic transitions and exact candidate filtering let lookahead compute the expected number of guesses directly, while a function approximator must estimate this quantity from sampled trajectories. The DQN's factorized output further constrains the hypothesis class: word value is modeled as a sum of letter-position contributions, which cannot represent cross-position interactions such as the "qu" bigram. Rollout avoids this by simulating under the true transition dynamics, achieving performance within 1.6% of optimal.

5.2 Expert Guidance and Imitation

The improvement from DQN v1 to v2 shows that learning is most effective when paired with structured prior knowledge. Tabular Q-learning works by learning over a small discrete strategy space, and DQN v2 works by taking guidance from a strong teacher policy. In both cases, embedding domain structure into the learning process reduces sample complexity.

This aligns with Ross et al. [8], who show that policies trained purely via supervised imitation suffer compounding errors that grow quadratically with the decision horizon, while training under the learner's own state distribution with expert guidance yields linear error growth. DQN v2 follows this principle by replacing random exploration with teacher-informed actions, concentrating training data on states the agent actually encounters during play.

5.3 Limitations

Several limitations apply to these results. Evaluation uses a closed 2,315-word vocabulary, unlike the official Wordle setting where guesses may be drawn from a larger dictionary, so comparisons are internal and not directly comparable to prior work. The deployed DQN solver also applies inference-time constraints not learned during training: single-candidate states return the remaining word directly, and previously guessed words are masked. These constraints improve stability, but reported performance reflects a combination of learned policy and external control logic rather than a fully end-to-end model.

Training scale is another limitation. The DQN models were trained for 10,000-20,000 episodes, several orders of magnitude fewer than the millions of games used by Ho [3], which restricts the model's ability to explore the state space and learn stable value estimates. The reward function is also misaligned with the true objective of minimizing expected guesses: it does not penalize weak early guesses directly, and the teacher reward is binary rather than a graded signal reflecting how close a non-matching action came to the teacher's choice.

DQN v2's performance is further bounded by its teacher, preventing it from exceeding rollout-based performance. All reinforcement learning results are based on a single training run, so variance across random seeds is not characterized.

5.4 Future Work

Several directions could extend this work. Evaluation should be repeated under the standard separated-vocabulary benchmark to enable direct comparison with prior results, and multiple training runs with different random seeds would characterize variance across seeds. Comparing DQN v2 against a pure behavioral cloning baseline would help isolate the contribution of reinforcement learning relative to imitation.

Other improvements include richer model classes such as architectures with full-word action representations, and a reward function that directly reflects candidate set reduction. Longer training or better exploration may also let DQN v2 surpass its teacher. Policy-gradient methods such as A2C, which Ho [3] found more effective at scale, are a promising direction, especially when combined with teacher-guided exploration.

6 CONCLUSION

This project compared six solver paradigms for Wordle under a unified exhaustive benchmark. The results establish a clear hierarchy: rollout planning (3.477) > frequency heuristics (3.575) > minimax search (3.644) > tabular meta-policy RL (3.651) > teacher-guided deep RL (3.678) >> pure deep RL (4.582), with all planning and heuristic methods achieving 100% win rates.

The motivating hypothesis that well-designed heuristics outperform unguided machine learning in this domain is confirmed. The improvement from DQN v1 to v2 shows that the gap is driven primarily by exploration and training signal rather than model capacity. The rollout solver's discovery of only 331 reachable states under near-optimal play further indicates that despite NP-hard worst-case complexity, the effective state space is highly constrained, favoring planning methods that exploit exact transitions and memoization.

The teacher-guided DQN also shows that learned methods can approach planning-level performance when trained on high-quality state-action data. In structured, deterministic environments such as Wordle, the most effective approaches may be hybrid methods that combine explicit planning with learned approximations.

ACKNOWLEDGMENTS

This project was developed as part of IT 448 (Graduate Machine Learning) at Illinois State University, Spring 2026. The original *Wordmaster Master* solver was developed in 2022. Source code is available at *https://github.com/jhffmn82/wordle-ML-project*, and an interactive demonstration is hosted at *https://jhffmn82-wordle-ml.hf.space*.

REFERENCES

[1] Daniel Lokshtanov and Bernardo Subercaseaux. 2022. Wordle Is NP-Hard. In Proceedings of the 11th International Conference on Fun with Algorithms (FUN 2022). Leibniz International Proceedings in Informatics (LIPIcs), Vol. 226, Article 19, 19:1–19:8. https://doi.org/10.4230/LIPIcs.FUN.2022.19

[2] Dimitris Bertsimas and Alex Paskov. 2025. An Exact Solution to Wordle. Operations Research. https://doi.org/10.1287/opre.2022.0434

[3] Andrew Ho. 2022. Solving Wordle with Reinforcement Learning. https://andrewkho.github.io/wordle-solver/

[4] Chao-Lin Liu. 2022. Using Wordle for Learning to Design and Compare Strategies. In Proceedings of the 2022 IEEE Conference on Games (CoG 2022), 465–472. https://doi.org/10.1109/CoG51982.2022.9893585

[5] Siddhant Bhambri, Amrita Bhattacharjee, and Dimitri P. Bertsekas. 2023. Playing Wordle Using an Online Rollout Algorithm for Deterministic POMDPs. In Proceedings of the 2023 IEEE Conference on Games (CoG 2023). (Extended version available at https://arxiv.org/abs/2211.10298)

[6] Benton J. Anderson and Jesse G. Meyer. 2022. Finding the Optimal Human Strategy for Wordle Using Maximum Correct Letter Probabilities and Reinforcement Learning. arXiv:2202.00557. https://arxiv.org/abs/2202.00557

[7] Jiaqi Weng and Chunlin Feng. 2023. Prediction Model for Wordle Game Results with High Robustness. arXiv:2309.14250. https://arxiv.org/abs/2309.14250

[8] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS 2011).

[9] Volodymyr Mnih et al. 2015. Human-level control through deep reinforcement learning. Nature 518, 7540 (2015), 529–533. https://doi.org/10.1038/nature14236

[10] Yu-Chun Chen, Hsiao-En Kuan, Yu-Shiang Lu, Tzu-Chi Chen, and I-Chen Wu. 2022. Entropy-Based Two-Phase Optimization Algorithm for Solving Wordle-like Games. In Proceedings of the 2022 International Conference on Technologies and Applications of Artificial Intelligence (TAAI 2022). https://doi.org/10.1109/TAAI57707.2022.00014